



Digital Design Verification
Shell Scripting
Lab Manual # 27

Release: 1.0

Date: 7- July -2024

NUST Chip Design Centre (NCDC), Islamabad, Pakistan



Copyrights ©, NUST Chip Design Centre (NCDC). All Rights Reserved. This document is prepared by NCDC and is for intended recipients only. It is not allowed to copy, modify, distribute or share, in part or full, without the consent of NCDC officials.

Revision History

Revision Number	Revision Date	Revision By	Nature of Revision	Approved By
1.0	07/07/2024	Muhammad Bilal	Complete manual	Musaddiq Hussain
1.1	21/05/2025	Muhammad Bilal	Manual Updated	Musaddiq Hussain



Contents

Objective.....	5
Tools and Technologies.....	5
Submission Guidelines.....	5
Overview.....	6
Task 1: Sorting an array using Bubble Sort	10
Task 2: Random Password Generator.....	10
Task 3: Git init and status	11
Task 4: File Type Segregation and Archiving	11
Task 5: Executable Finder and Logger	11
References	11



Objective

The objective of this lab is to

- Understanding the bash scripting and automating the repetitive tasks with it.

Tools and Technologies

- Ubuntu/Redhat

Submission Guidelines

You will need to submit the script files and a single report which includes the screenshots of the tested script.



Overview

Arguments:

\$0 - Name of the script

\$1 to \$9 - Arguments to the script. \$1 is the first argument and so on.

\$@ - All the arguments

\$# - Number of arguments

\$? - Return code of the previous command

\$\$ - Process identification number (PID) for the current script

!! - Entire last command, including arguments. A common pattern is to execute a command only for it to fail due to missing permissions; you can quickly re-execute the command with sudo by doing sudo !!

\$_ - Last argument from the last command. If you are in an interactive shell, you can also quickly get this value by typing Esc followed by . or Alt+.

Command substitution:

To get the output of a command as a variable (command substitution).

\$(CMD) it will execute CMD, get the output of the command and substitute it in place. For example, **for file in \$(ls)**

Process substitution:

<(CMD) will execute CMD and place the output in a temporary file and substitute the **<()** with that file's name. This is useful when commands expect values to be passed by file instead of by STDIN.

For example, **diff <(ls foo) <(ls bar)** will show differences between files in directories foo and bar.

Wildcards -

Wildcard matching, you can use **?** and ***** to match one or any amount of characters respectively.

For instance, given files foo, foo1, foo2, foo10 and bar, the command **rm foo?** will delete foo1 and foo2 whereas **rm foo*** will delete all but bar.



Curly braces {} -

Whenever you have a common substring in a series of commands, you can use curly braces for bash to expand this automatically. e.g

```
mkdir foo bar
```

```
# This creates files foo/a, foo/b, ... foo/h, bar/a, bar/b, ... bar/h
```

```
touch {foo,bar}/{a..h}
```

Getting User Input

```
#!/bin/bash
```

```
read -p "What is your name? " name
```

```
echo "Hi there $name"
```

```
echo "Welcome to NCDC!"
```

Alias:

In Linux, an alias is a user-defined shorthand for a longer command or sequence of commands. These aliases can be created and customized according to user preferences, making the command-line interface more user-friendly.

```
alias <your CMD>="<Actual Command>"
```

Find Directories and Files:

```
# Find all directories named src
```

```
find . -name src -type d
```

```
# Find all python files that have a folder named test in their path
```

```
find . -path '*/test/*.py' -type f
```

```
# Find all files modified in the last day
```

```
find . -mtime -1
```

```
# Find all zip files with size in range 500k to 10M
```

```
find . -size +500k -size -10M -name '*.tar.gz'
```



Zip Commands

The zip command is used to create compressed archives of files and directories. It is useful for reducing the size of files and directories for storage or transfer.

```
# Zip all files in the current directory into archive.zip
zip archive.zip *

# Recursively zip the contents of the my_directory into my_directory.zip
zip -r my_directory.zip my_directory

# Add additional_file.txt to existing_archive.zip
zip existing_archive.zip additional_file.txt

# Zip all files in the current directory except those ending in .log
zip -r archive.zip * -x '*.log'
```

Bash Scripting

"Shell" is a broad term that refers to any program that provides a command-line interface, "Bash" is a specific type of shell that is widely used in Unix/Linux systems.

Arrays:

```
my_array=("value 1" "value 2" "value 3" "value 4")
echo ${my_array[1]}
echo ${my_array[-1]} # return the last element: value 4
echo ${my_array[@]} # All the arguments
echo ${#my_array[@]} # No of arguments in array
```

Conditional Statements:

a. If else

```
if [[ some_test ]]
then
    <commands>
fi
```




Types:

1. File Expression:

`[[ -a ${file} ]]` # True if file exists.

`[[ -c ${file} ]]` # True if file exists and is a character special file.

2. String expressions:

`[[ ${string1} == ${string2} ]]` # True if the strings are equal.

3. Arithmetic Operation:

`[[ ${arg1} -ne ${arg2} ]]` # Returns true if the numbers are not equal

4. Exit Status Operations:

`[[ $? -eq 0 ]]` # returns true if the command was successful without any errors

Bash implements many comparisons of this sort - you can find a detailed list in the **manpage** for **test**.

b. Switch Case:

```
case $some_variable in
  pattern_1)          # single condition
    commands
    ;;
  pattern_2| pattern_3) # multiple OR case
    commands
    ;;
  *)                  # * for default case
    default commands
    ;;
esac
```

c. Loops

1. For Loop

```
for var in ${list}
do
  your_commands
done
```



2. **While:**

```
while [ your_condition ]  
  
do  
  
    your_commands  
  
done
```

3. **Do While (Untill)**

```
until [[ your_condition ]]  
  
do  
  
    your_commands  
  
done
```

break [n] # break out of nth nested loop

continue [n] #n-th enclosing loop is resumed

d. **Functions:**

```
function function_name() {  
    your_commands  
}
```

OR

```
function_name() {  
    your_commands  
}
```

Task 1: Sorting an array using Bubble Sort

Write a script that sorts an array using Bubble Sort algorithm. The array can be hard-coded. Submit the script file and a document that briefly explains the script. Also add a screenshot of your terminal output in the document.

Task 2: Random Password Generator

Write a script that generates the password of the length defined by the user input. It should consist of **alpha numeric special characters**. Submit the script file and a document that briefly explains the script. Also add a screenshot of your terminal output in the document.



Task 3: Git init and status

The goal of this task is to simulate **Git init** and **Git status** commands. In this task we want you to create backup of your current directory when “**init**” function is called and write a function called “**status**” that prints the difference between your backup and updated files.

Task 4: File Type Segregation and Archiving

Write a script that will segregate files based on their extensions within a given directory (**files/**). Create separate subdirectories in “**files**” folder for .pdf, .c, and .h files and move the files into their respective directories. After segregation, create a zip archive of each of these new subdirectories and place the archives in the “**files**” folder.

Task 5: Executable Finder and Logger

Write a script that will find all executable files in the subdirectories of the source/ directory, print their names, and log them to a file named **executables.log** in the source/ directory. The log file should have the same format as the printed output, listing each subdirectory followed by its executable files.

Submit the script file and a document that briefly explains the script. Also add a screenshot of your terminal output in the document.

References

1. Course overview + the shell
<https://missing.csail.mit.edu/2020/course-shell/>
2. Shell Tools and Scripting
<https://missing.csail.mit.edu/2020/shell-tools/>
3. Command-line Environment
<https://missing.csail.mit.edu/2020/command-line/>
4. CALIFORNIA STATE UNIVERSITY: GISC 605
<https://gisc605.alexpakalniskis.com/lab-exercises/4-getting-started-with-bash-scripting>
5. ShanghaiTech University | Computer Architecture:
<https://robotics.shanghaitech.edu.cn/courses/ca/19s/labs/12/>
6. University of California San Diego:
http://ieng6.ucsd.edu/~cs15x/labs/lab6_sbfqkf/index.html
7. Clemson University(CPSC 1070 Lab 11):
<https://people.computing.clemson.edu/~dhouse/courses/1070/labs/>